altairsim — Developer Guide

2026-07-27 · e86431d

Building it

There are no dependencies. A C++20 compiler and CMake ≥ 3.20 is the entire list. The TOML parser, the JSON encoder and the line editor are all in the tree, so a fresh clone builds with nothing to download.

```
git clone https://github.com/deltecent/altairsim.git
cd altairsim
cmake -S . -B build && cmake --build build -j
ctest --test-dir build -LE slow      # drop -LE slow for the full 8080 exerciser
./build/altairsim                    # the default machine
```

If CMake is unfamiliar, the two `cmake` calls are two distinct steps — **configure**, then **build** — and you run both in that order:

- **`cmake -S . -B build`** — *configure.* Reads `CMakeLists.txt` in the source tree (`-S .`, the current directory) and writes the generated build files into a fresh `build/` directory (`-B build`). It downloads nothing and compiles nothing; it just works out *how* to build. You rerun it only when `CMakeLists.txt` itself changes — and even then the build step reruns it for you.
- **`cmake --build build -j`** — *build.* Compiles what the configure step laid out under `build/`. `-j` runs the compiles in parallel across all CPU cores (drop it for a serial build, or `-j4` to cap the jobs). This is the command you repeat after editing code.
- **`ctest --test-dir build -LE slow`** — run the tests that the build produced. `--test-dir build` points at the same directory; `-LE` is *label-exclude*, so `-LE slow` runs everything **except** tests tagged `slow` (the ~10-minute 8080 exerciser). Drop it to run those too.

Everything lands in `build/`, which is disposable: `rm -rf build` and rerun the configure step for a clean slate.

That is not a boast about minimalism for its own sake. It is a property worth defending: the day this needs a package manager to build is the day it stops being something a person can pick up in ten years and compile.

After a `git pull`, the commands do not change. `cmake -S . -B build && cmake --build build` again — there is no separate incremental procedure and no need to delete `build/`. New `.cpp` files can't be missed because sources are listed in `CMakeLists.txt` rather than globbed, so adding one edits `CMakeLists.txt` and forces a reconfigure; `roms/` and `machines/` *are* globbed but use `CONFIGURE_DEPENDS`, so they re-glob on every build. The build directory does cache absolute paths, though, so **rename the checkout or switch compilers and you must `rm -rf build`** — it fails with a loud `CMakeCache.txt directory is different` error, not a silent wrong answer. If a build breaks — or a test that used to pass fails — immediately after a pull, suspect a stale `build/` before suspecting the code: a `rm -rf build` and a clean reconfigure is

the first thing to rule out, because a half-rebuilt object can walk an already-fixed bug back in. `docs/building-linux.md` §6 has the details.

The optional video backend (SDL3)

Still no *required* dependency. The graphics boards — the VDM-1, and the Dazzler to follow — draw into a host `Display` (`src/host/display.h`). That interface has two backends: a headless `NullDisplay` (draws into memory, shows nothing) that is always available, and an `SdlDisplay` (a real window) compiled **only when SDL3 is found**. So a plain `cmake -S . -B build` on a machine with no SDL3 still configures, builds, and passes every test — the video boards are simply headless, which is exactly what CI and a scripted run want.

To get a window, install SDL3 and reconfigure:

```
# macOS
brew install sdl3
# Debian/Ubuntu (needs a recent SDL3 package)
sudo apt install libsdl3-dev
# vcpkg (any OS)
vcpkg install sdl3

cmake -S . -B build          # reconfigure -- watch the status line
```

The configure step reports which backend it chose:

```
-- SDL3 found -- video boards enabled (windowed)
        …or…
-- SDL3 not found -- video boards build headless (null display). Install SDL3 (see docs/devguide)
for a window.
```

CMake auto-detects with `find_package(SDL3 CONFIG)`; found → it compiles `src/host/display_sdl.cpp`, links `SDL3::SDL3`, and defines `ALTAIRSIM_ENABLE_SDL`. Force a headless build even where SDL3 is installed with **`-DALTAIRSIM_ENABLE_SDL=OFF`**. That flag is what a macOS *universal* build needs, because a Homebrew SDL3 is single-arch and cannot link into an `x86_64;arm64` fat binary. **CI passes it nowhere. One leg — macOS — installs SDL3 from Homebrew and builds native `arm64`, so it is the only place `display_sdl.cpp` is compiled at all**, and the workflow fails that leg if it comes up headless. Linux and Windows take the not-found path above and build against the null display. Only `display_sdl.cpp` and the composition root (`src/main.cpp`) are macro-gated; the boards themselves `#include` no SDL and compile in every configuration. Then:

```
altairsim vdm1               # a VDM-1 with a banner-drawing demo (roms/VDM1DEMO)
```

What actually got built

Built and tested on Linux, macOS, and Windows. The code is written to be portable — C++20, no dependencies, and every OS difference confined to `src/platform/` behind a header with zero conditionals — and that portability is now proven, not asserted: Linux (Ubuntu/GCC), macOS on Apple Silicon (and on Intel, built natively there rather than in CI), and Windows on MSVC all build and pass the suite. The Windows platform layer, once merely written, is field-proven.

CI runs the suite on every push. GitHub Actions builds and tests on all three platforms — Linux, macOS, and Windows are each a required check — so a regression on any of them shows up before it merges. The tests still run locally the same way, when someone types `ctest`.

Each of those jobs uploads the binary it built, so a green run leaves three executables on GitHub — including the two you cannot produce on your own machine. To fetch them:

```
tools/fetch-ci-binaries.sh          # newest CI run on the current branch
tools/fetch-ci-binaries.sh 42       # ...on PR 42
```

It **waits** for the run if it is still going and refuses to download from a red one, which makes it a reasonable last step before merging: three files means three platforms passed. They land in `./artifacts` (git-ignored, replaced on every fetch) with the executable bit restored, since the artifact zip does not carry POSIX modes. Nothing in the build or the tests reads from there — these are CI's binaries, kept for running or handing to someone, not a build output of this tree.

The tests

```
ctest --test-dir build -LE slow     # unit + acceptance. About 30 seconds.
ctest --test-dir build              # ...plus 8080EXM, the full exerciser.
ctest --test-dir build -L hw        # modem control, against a real null-modem cable.
```

The acceptance tests are not unit tests. They **boot period software on the whole machine through the real CLI** and check what lands on the terminal — 4K and 8K BASIC off a cassette, MITS Programming System II polled and interrupt-driven, CP/M off a minidisk. Several ship with a **negative control**: the same script against a machine that ought to *fail*, marked `WILL_FAIL`. If a control ever passes, the test it guards was passing for the wrong reason and is worthless. That is the only reason to believe any of them.

The CPU exercisers are gated three ways, on purpose. `cpu-8080exm` is ~2.9 billion instructions and `cpu-zexdoc` / `cpu-zexall` ~5.8 billion each, so they are labelled `slow` and excluded from the per-push matrix. `cpu-exerciser.yml` runs them on one Linux runner *only when CPU/ISA code changes* — an opcode bug is a logic bug and shows up on any host, so one

architecture is enough to catch it, and paying for three would be paying three times for the same answer. `cpu-exerciser-release.yml` then runs them on **all three platforms** at a release tag (or on demand), because the one thing that is *not* architecture-independent is the compiler: the cores are full of shifts, masks and half-carry arithmetic, and until a release nothing has ever driven billions of instructions through the MSVC build.

The hardware tests (`-L hw`) run against an actual null-modem cable between two USB serial ports, because a claim about a cable deserves a cable. They are opt-in, pointed at your ports with `ALTAIR_SERIAL_A` / `ALTAIR_SERIAL_B`, and they **skip loudly** when the hardware is absent — a hardware test that quietly passes with no hardware is a green tick that means nothing.

The documentation is part of the build

Two documents come out of this tree:

Target	Is
User Manual (`docs/manual/`)	Ships in the package. **Self-contained** — it may not name a single file the reader does not have. No `src/`, no CMake, no `DESIGN.md`.
Developer Guide (`docs/devguide/`)	This. Repo only. Free to talk about the source, because you cannot write a board without it.

Some of the manual is **generated from the binary**, and this matters if you touch a board:

```
cmake --build build --target docs-reference
```

That rewrites `docs/manual/ref/*.md` — the command dictionary, the per-board parameter tables, the machine list — by walking `Board::properties()` and the `CommandDef` table. Those files are committed, and **a test (`docs-reference`) fails if they are stale.** So if you add a property, change a default, or reword a `HELP` string, run it and commit the result alongside your change.

This is not a docs chore bolted on at the end. A board's properties **are** its TOML schema; a hand-written parameter table would be a second schema, and the first draft of one in this very project got three of the memory card's eight defaults wrong. The reference is printed rather than retyped for the same reason the rest of the program has one source of truth for anything.

The PDFs need `pandoc` and any Chromium-based browser, and are built by `tools/build-docs.sh`. Neither tool is a build dependency, and neither goes anywhere near the simulator.

Theory of operation

This chapter is about what is actually going on inside the machine. It assumes you have the source, and it names files: `src/core/bus.h`, `src/core/board.h`, `src/core/value.h`, `src/core/machine.h`, and `DESIGN.md`, which is the document all of this was argued out in.

The next chapter builds a board — a lamp latch on port `FF`. Everything here is what that board is standing on.

The whole architecture, in one sentence

> **Boards respond to bus cycles. The CPU originates them.**

That is not a slogan. It is the shape of the code, and everything else in this chapter is a consequence of it.

The two concepts are separate types, and they live together in `src/core/bus.h` rather than with the CPU:

```
class BusMaster {
public:
    virtual StepResult step(Bus&) = 0;
};
```

A CPU card is **both**: `Cpu8080Board : public Board, public BusMaster`. It is a card you can pull out with your hand (`BOARDS REMOVE cpu0` works, and a machine with no processor in it is a real machine — it is the one the monitor drove before the 8080 existed), and it is also the thing that drives cycles onto the backplane.

The payoff is DMA, and it is free. S-100 has `pHOLD`/`pHLDA` precisely because a backplane can have more than one master. A DMA card — a disk controller stealing cycles, a Dazzler — is a `Board` that *becomes* a `BusMaster` when it is granted the bus, and it drives the very same cycles through the very same interface the CPU uses. **DMA is never a special path bolted onto the bus.** If you find yourself writing one, the model has already gone wrong.

It is also why `BREAK MEM W 0100` catches a DMA write. A cycle is a cycle; nothing on the backplane records who originated it. There is deliberately **no `origin` field** on `BusCycle`, and `src/core/bus.h` says why: a real backplane cycle carries no such tag, which is exactly why a front-panel `DEPOSIT` is indistinguishable from a CPU write, and why a real ROM ignores both.

The bus carries signals. It does not invent behavior.

The second rule, and the one the bus header exists to enforce:

> The bus arbitrates no overlay, vectors no interrupt, knows no bank, and has never heard of ROM.

It picks no winner between two cards. It does not hand the CPU an interrupt vector. It does not route reads to one board and writes to another. Every one of those lives in a board.

When you are tempted to add `if (board is a ROM)` to `Bus`, **you have found a bug in your board instead.** There is exactly one thing the bus does that no board does, and §4.6.1 of DESIGN.md pins it down: it supplies `0xFF` when nobody answered. That is the entire bus/board overlap, and it stays that size.

A bus cycle, end to end

```
enum class Cycle { MemRead, MemWrite, IoRead, IoWrite, IntAck };

struct BusCycle {
    Cycle type = Cycle::MemRead;
    uint16_t addr = 0;   // memory address; for I/O the port is addr & 0xFF
    uint8_t data = 0;
    bool phantom = false;

    uint8_t port() const { return (uint8_t)(addr & 0xFF); }
    bool isWrite() const { return type == Cycle::MemWrite || type == Cycle::IoWrite; }
};
```

Five cycle types. `IntAck` is one of them, and that is load-bearing — see interrupts, below.

`data` is valid on a write. **On a read it is zero while the cycle is in flight** — nobody has driven the bus yet when `decodes()` and `read()` are asked, so there is nothing honest to put there — and it is **back-filled** with the byte that came back (a board's, or the floating bus's `0xFF`) before `snoop()` and the observers see it. `Bus::settle()`.

The bus runs each cycle in three passes:

Pass	What it does
1	Ask every board whether it pulls `PHANTOM*` → set `BusCycle::phantom`
2	Ask who `decodes()` this cycle. Exactly one should answer. Nobody → `0xFF`.
3	`settle()`: show the completed cycle to every board that asked to see it.

Carrying a signal, running a decode, and letting the cards watch. **No decisions.**

The decode is cached, because on real hardware it is wired

A card's address decoder is combinational logic — a PAL, a row of gates — wired to the address lines and to the status strobes `sMEMR`, `sINP`, `sOUT`. It does not *answer a question* once per cycle. It **settles**, and it only changes when something latches: a bank strap, `PHANTOM*`, a card pulled from the backplane.

So the bus asks the same questions of the same boards in slot order, and asks them **once**, storing the answer in four tables:

```
struct Slot {
    Board* who = nullptr;   // the single board that drives. null: nobody -> floats 0xFF
    bool phantom = false;   // PHANTOM* as resolved for this page and this cycle class
    bool slow = false;      // more than one driver. Contention: take the exact path.
};

Slot memRead_[256], memWrite_[256];
Slot ioRead_[256], ioWrite_[256];
```

One entry per **256-byte page** for memory, one per **port** for I/O, per cycle class. A decode is now a table lookup, not a walk over every board in the machine.

The board still owns the entire decision. The bus still invents nothing. It just stopped asking sixty-five thousand times a second — and stopped asking cards questions they are not even wired for. An I/O-only 2SIO used to be asked to decode every memory read, and its first act was to throw the question away. **A real 2SIO has no connection to the memory read strobe. It is not in that conversation.**

Three consequences follow, and all three are things you can get wrong.

`decodes()` must be pure

```
virtual bool assertsPhantom(const BusCycle&) const { return false; }
virtual bool decodes(const BusCycle&) const { return false; }
```

Same board state, same cycle → same answer, and no side effects. Both of these are combinational. The bus calls them several times per cycle — once to resolve the signal, again inside each board's own decode — and then it *caches the result*.

A `decodes()` with a side effect in it is a bug that will not surface for a month. It will fire the right number of times on the day you write it and the wrong number of times forever after, because the number of times the bus asks is an implementation detail of a cache. Latch nothing here. The clocked half is `snoop()`.

If your decode changes, say so

```
void decodeChanged();   // "my decode just changed" -- sets a dirty flag on the bus
```

A bank strap moved. A `PHANTOM*` jumper moved. A chip came out of a socket. The card went `enabled = false`. **Call `decodeChanged()`.** Forget, and the tables go stale and the machine lies quietly, which is the worst failure mode there is.

You mostly get this for free: `setProperty()` — the one path by which any property is *ever* set, from `SET`, from TOML, from `BOARDS ADD`, from MCP — calls `configChanged()` on the board after every successful set, and the default `configChanged()` calls `decodeChanged()` and `intChanged()`. You call it by hand for changes that do not come through a property: a guest `OUT` that moves a bank strap, a boot ROM switching itself out.

And it is not left to trust. `Bus::setVerify(true)` re-derives the decode the slow way on every single cycle and screams the moment it disagrees with the table. The unit suites run with it on permanently; the 8080 validation gate runs with it over 2.9 billion instructions. **It is a proof, not a path** — it is slower than the code it replaced, and that is fine.

The bus routes by CYCLE TYPE, not just by address

Four tables, not one. That is not an optimization detail — it is the model:

A card is wired to `sMEMR`, `sINP` and `sOUT` separately. A ROM famously does not decode a write *at all*; it does not reject the write, or ignore it, or log it — it never answers the cycle. So "who answers here" has a **different answer for a read than for a write**, and that falls out of the model rather than being bolted onto it.

The consequence you will use immediately: **one card can own `IN FF` while a completely different card owns `OUT FF`, with no contention whatsoever.** That is not a trick. It is what the Altair front panel actually does. `src/boards/mits-frontpanel.h` decodes exactly this and nothing else:

```
bool decodes(const BusCycle& c) const override {
    return enabled_ && c.type == Cycle::IoRead && c.port() == 0xFF;
}
```

Its sense-switch buffers are gated with `sINP`. There is no `sOUT` anywhere near them, so an `OUT 0FFH` **is not the panel's** — it goes unclaimed and the backplane discards the byte, which is precisely what the hardware does with it. Port `FF` output is therefore a free space on a real Altair, and it is where the next chapter's lamp board lives.

`decodeIsPageUniform()` — when a card decodes a low address line

```
virtual bool decodeIsPageUniform() const { return true; }
```

Memory decode is cached one entry per 256-byte page, and that is a **contract on `decodes()`**. Nearly every card can keep it: S-100 memory decoding comes off the *high* address lines, and a card selected at 1K or 4K granularity answers a whole page or none of it.

A card that decodes a low address line says `false`. The Tarbell is the real one — its PROM and its `PHANTOM*` are gated by **A5**, so it decodes `0000-001F` and *not* `0020-003F`: two different answers inside page 0. Say `false` and the bus probes every address of every page you might be in, and any page whose answer is not uniform is served by the exact, uncached two-pass path. **You lose nothing but the cache, and only on the pages you actually touch.**

`snoop()` — the clocked half, and the only place you may latch

```
virtual bool wantsSnoop() const { return false; }
virtual void snoop(const BusCycle&) {}
```

Every board sees every cycle. That is what a backplane IS — the address bus is not addressed *to* anyone, it is simply present, and any card may watch it whether or not it answers.

`snoop()` is called exactly **once per cycle, after it completes**, with `data` back-filled. It is the **only** place a board may latch what it saw. It is opt-in via `wantsSnoop()`, because calling a do-nothing virtual on every card on every cycle was pure ceremony.

The front panel says yes, and it is the clearest example of why the hook exists: **its lamps are wired to the backplane.** Not a metaphor — an LED on a bus line sees exactly the cycles `snoop()` hands you, and `FrontPanelBoard::snoop()` is the only place `addrLeds_`, `dataLeds_` and `status_` are written. The lamps show whatever went by, and at 2 MHz that is a blur. It was a blur in 1975; the MITS manual says so.

The bus is not *notifying* anyone here. The cycle was on the backplane the whole time and they could all see it. This is only where we let them latch it.

`peek()` — look without touching

```
virtual bool peek(uint16_t addr, uint8_t& out) const { return false; }
```

A `read()` may CONSUME. An `IN` from a UART's data port takes the byte and the guest never sees it again. So `DISASM`, `TRACE` and the debugger's register display are built on `peek()`, never on a read — a disassembler that ate the console's input the first time someone disassembled a page with a UART mapped into it would be a debugger you could not trust.

`peek()` runs the same decode, `PHANTOM*` and all, so a shadowed board is invisible to it exactly as it is to a real read. It just never strobes anybody.

A board that cannot answer without side effects returns `false`, **and that is an honest answer, not a failure**: the byte on a real bus is only defined *during* a cycle. The caller shows `FF`, which is what the bus would have floated to.

The floating bus — one rule, three consequences

> **If no board drives the bus, it floats high. Every read of an unmapped address or port returns `0xFF`. Writes go nowhere.**

One rule. Three things that would otherwise each need a special case fall straight out of it:

Situation	What happens	Why it matters
Unpopulated memory	reads `0xFF`	Period software **sizes memory by reading**. Return `0x00` and every machine looks like it has 64K, and CP/M builds itself wrong.
Unmapped I/O port	reads `0xFF`	A guest probing for a board that is not there gets the answer real hardware gives it.
`IntAck` with nobody driving	reads `0xFF` = `RST 7`	The famous one. See below.

With no vector card in the machine: a board pulls pin 73, the CPU runs an `IntAck` cycle, nobody claims it, the bus floats, the CPU reads `FF` — and `FF` is `RST 7`. That is not a fallback anybody coded. It is what a real Altair does, and it is why the PMMI's factory jumper straight to pin 73 yields `RST 7` with no vector logic anywhere in the machine.

Model the floating bus honestly once and all three are free. Fake any one of them and you will fake the other two differently.

Therefore no board may ever manufacture `0xFF`, and in particular no board may seed its own store with it. A RAM chip does not power up holding `FF`; it powers up holding whatever it feels like, which is what `fill = random` is for. Seed a board's store with `FF` and `DUMP` shows `FF` for a card whose RAM is fine, `FF` for a card whose RAM was never filled, and `FF` for a card that **isn't in the machine**. One symptom, three causes. The moment a board can produce `FF`, the only signal the bus has stops being a signal. `tests/test_boundary.cpp` enforces it.

Interrupts

Two wires' worth of interface, and the same rules apply to both.

```
virtual bool    assertsInt() const { return false; }   // pINT, S-100 pin 73
virtual uint8_t assertsVi()  const { return 0; }       // VI0-VI7, pins 4-11, as a BITMASK
virtual bool    watchesVi()  const { return false; }   // an 88-VI says yes; nothing else does
virtual int     intWinner()  const { return -1; }      // ...and only it has an opinion
void            intChanged();                          // "my pin may have moved"
```

An interrupt is a LEVEL, not an event

A UART with a character waiting and its interrupt jumper installed says `true`, **and keeps saying `true` until the guest reads the character.** There is no queue, so a board cannot "lose" an interrupt — there was never a queue to lose it from.

Both of these are **combinational and pure**, exactly like `decodes()`. `assertsInt()` reports the settled state of a pin, computed from the state of the chip and nothing else. It does not advance a receiver, take a byte off a line, or do any work the guest has not paid for.

It used to do all three, and there was a whole section of DESIGN.md defending it. The 6850 has to notice a character has finished arriving, which happens on the chip's own clock with no help from the CPU — and being asked `assertsInt()` was the only thing that ever woke the card up. **The poll was serving as the card's clock.** That work moved to where it belongs: a `Clock` deadline the card sets for itself, and `pump()`. Read §4.4.1 of DESIGN.md before you are tempted to do work inside one of these.

`assertsVi()` is a bitmask, not a level

Bit *n* means "I am pulling VI*n*". It is a mask and not a level because **a card can pull two lines at once**: the 88-SIO straps its input device and its output device independently, and the manual is explicit that they may sit at different priorities. Both can be asking in the same instant — a

character has arrived *and* the transmitter has gone empty. A single `int` level would have to pick one and drop the other, silently, and only when both fired. That is the worst possible way to lose an interrupt.

Where a card's request is soldered is a **bus strap**, spelled the same on every card in the machine (`src/core/board.h`):

```
enum class IrqJumper { None, Int, Vi0, Vi1, Vi2, Vi3, Vi4, Vi5, Vi6, Vi7 };
Property irqJumperProperty(std::string name, std::string help, IrqJumper& j);
```

Use `irqJumperProperty()` and your board gets the same ten choices, the same spelling, the same tab completion, and a `SHOW BUS IRQ` line — and a card with *two* straps gets them both for free.

The bus does not poll. It keeps the wire.

`Bus::intPending()` used to walk the backplane and ask every card `assertsInt()`, **once per instruction** — sixty million times a second, to compute a boolean that changes maybe a thousand times a second. It was the single largest per-instruction cost left in the simulator once the decode was cached, and it *grew with every card you added*.

Now the board **pulls the pin** and the bus keeps a running wire-OR: `intCount_` for pin 73, and `viCount_[8]` plus a `viMask_` for the eight VI lines. Reading pin 73 is one integer test, flat in the number of cards.

The two hard-won rules turn out to be one rule:

	combinational, pure	**"it moved"**	**what the bus keeps**
address decode	`decodes()`	`decodeChanged()`	a page table
interrupt	`assertsInt()`	`intChanged()`	a wire-OR count

> **A board's outputs are pure functions of its state. When its state changes, it says so. The bus caches the rest.**

Call `intChanged()` after anything that could move the pin: a register written, a character taken off the line, a deadline coming due, a jumper moved. A spurious call costs a virtual call. **A missing one hangs the guest forever**, waiting for an interrupt that already happened — and it presents as "*the emulator locks up sometimes*", which is worth a week of anyone's life.

So it is not left to trust either. `Bus::setVerify(true)` re-derives the **whole** wire — pin 73 and all eight VI lines — from every board's `assertsInt()` / `assertsVi()` on every instruction, and

aborts the moment a board disagrees with the cache.

One call covers all nine wires on purpose. A board does not know which wire its jumper is in today, and a card that had to announce each wire separately would eventually forget one.

Where the vector comes from

Nowhere in the bus. **The vector comes from whoever claims the `IntAck` cycle — like any other cycle.**

That is the whole design. `watchesVi()` and `intWinner()` are the 88-VI's, and nothing else's: it watches the eight lines, applies its own priority and its own mask, drives pin 73 itself, and then claims `IntAck` and jams an `RST n` onto the data bus. It is an **ordinary board**. It has no special privileges, and neither will any new interrupt controller you invent — which is the point of the project.

`watchesVi()` exists because a card that *pulls* a VI line announces it, but the card *watching* those lines is a third party who was told nothing and whose own pin 73 has just gone stale. The bus calls `intChanged()` on each watcher when a VI line actually moves. `intWinner()` exists so `SHOW BUS IRQ` can say *which* line wins without the monitor knowing what an 88-VI is — the alternative was a `dynamic_cast` in the monitor, and **the monitor does not get to know about cards.**

Memory, ROM, and `PHANTOM*`

A memory card is **a list of regions**, not an address range: RAM here, a ROM socket there, an empty socket between them. One card can occupy two disjoint ranges, because a real one does. `SHOW BUS MAP` is per-*range*, not per-board, for that reason.

An empty socket decodes nothing. It does not read as zero — it floats to `FF`, like anything else nobody drives.

`PHANTOM*` is a signal a board pulls

`PHANTOM*` is **S-100 pin 67**, a line any board may pull low.

```
virtual bool assertsPhantom(const BusCycle&) const { return false; }
```

A boot ROM pulls it. A memory board **strapped to honour it** takes *itself* off the bus for that cycle — it returns `false` from its own `decodes()`. **The bus picks no winner.** The overlay is *emergent*: the ROM is the only board still answering, because the RAM switched itself off.

That is the entire overlay mechanism, and it is how a boot PROM shadows RAM at `FF00` and then gets out of the way (`setEnabled(false)`, usually as its last act, usually triggered by the boot code writing to the ROM card's own port — an ordinary `OUT` the card decodes).

Two things fall out of it that you must not re-derive by hand:

- **A shadow is not contention.** `Bus::respondersTo()` returns every board that *actually* decodes, so a phantomed-out board does not appear in it, and `SHOW BUS CONTENTION` stays quiet. Contention is decided *after* the phantom pass, on who is really driving. Two cards *registered* at one address is normal and correct — that is what a boot ROM over RAM **is**.
- **A card must not switch itself off with a signal it is itself driving.** The memory board's decode carries a `!assertsPhantom(c)` clause for exactly that, and its absence was a real bug in the default machine: the ROM pulls the pin, honours the pin, disappears, and reads back `FF` off its own floating bus.

`PHANTOM*` is a LEVEL, and the asserting card has no opinion about writes. **The read/write distinction lives on the *honouring* board** — which is why the memory card's jumper is `honors_phantom = none | read | all` and not a `bool`. `read` means *stop answering reads, keep answering writes*, so the byte lands in the RAM under the shadow. That is what lets a Tarbell bootstrap write the sector it is loading into the very RAM its own PROM is covering. (This paragraph used to say the exact opposite, in bold, and it was wrong — reasoned instead of sourced. Patrick read the schematic.)

`rawRead` / `rawWrite` / `rawSize` — the PROM burner

```
virtual size_t  rawSize() const { return 0; }
virtual uint8_t rawRead(size_t) const { return 0xFF; }
virtual bool    rawWrite(size_t, uint8_t) { return false; }
```

Straight into the card's backing store, **bypassing decode entirely**. Offsets are board-local, and the store may be far larger than 64K — a banked card's bank 3 simply *is* offset `0x30000`.

This is the PROM burner, and that is not a metaphor. It is how the operator writes a ROM the guest cannot, because **burning a PROM is not a bus operation on real hardware either. You pull the chip.**

Model it as a bus write instead and the bus would have to know *who originated a cycle* — which a real backplane cannot know, and which no board should ever have to ask.

Reflection is the keystone

```
virtual std::vector<Property> properties() = 0;

struct Property {
    std::string name;                   // "baud", "phantom", "honors_phantom"
    std::string help;                   // one line, shown by SHOW
    Kind kind = Kind::Int;              // Int | Bool | Str | Enum

    std::vector<std::string> choices;   // Kind::Enum -- also feeds tab completion
    long long min = 0, max = 0;         // Kind::Int; min==max means unbounded
    int radix = 10;                     // 16 for addresses, so SHOW reads right
    std::string unit;                   // "Hz", "bytes" -- display only
    bool irqJumper = false;

    std::function<Value()> get;
    std::function<bool(const Value&, std::string& err)> set;
};
```

`SET`, `SHOW`, the TOML loader, `CONFIG SAVE`, the MCP tool schemas, tab completion **and the manual's generated board reference** are all written **once**, against this. They know nothing board-specific.

There is no second schema anywhere. A board's TOML keys *are* its properties. A board added next year is configurable, scriptable, agent-drivable and documented **the day it lands**, and none of those six consumers changes a line.

The cost of that is that a property has to carry enough metadata to validate, render and describe itself. That is the whole trick, and it is why `radix` is there: `PORT=10` is port `0x10` and `BAUD=9600` is nine thousand six hundred, because the property said so.

Three things to get right when you write one.

A property with no setter is read-only. Live pin state, a derived value, a lamp. Leave `set` empty — **and that absence is the only signal any consumer has.** A setter that always refuses would stop a `SET`, and simultaneously fool `SHOW`, `CONFIG SAVE`, the MCP schema and the generated docs, all four of which read the *presence* of the function. Do not write one.

There is no config-time-only property. Every property can be set, always. You can only type at the prompt when the machine is stopped — that is the front panel's STOP switch, and there is no moment at which a `SET` races a running CPU. And on real hardware the gate would be a fiction anyway: a card being worked on sits on an **extender**, out where you can reach it, and its jumpers get moved with the power on.

`unitProperties()` is for a real sub-thing with its own settings.

```
virtual std::vector<Property> unitProperties(const std::string& unit) { return {}; }
```

The two halves of a 2SIO are **two independent 6850s** with their own crystals' worth of jumpers — independent baud rates, independent transforms — so they are units, and `SET sio0:a BAUD=9600` reaches one of them. Folding them into `properties()` as `a_baud` / `b_baud` would work for a 2SIO and fall apart on the first card with eight ports. Most boards return `{}` here, and that is fine.

A unit is a **name and a kind** (`UnitDef`, `UnitKind::{Disk, Rom, Serial, Tape, Cpu}`), never an index — so `MOUNT dj:drive0`, and mounting a disk image onto a serial port is an *error with a sentence*, not undefined behaviour that half-works. `units()` is the only list; `SHOW`, `MOUNT`, `CONNECT`, the MCP schemas and tab completion all read it, so they cannot disagree about what exists.

Reset is two different events

```
enum class Reset {
    PowerOn,  // POC* -- pin 76. Nothing in software can assert it; only the power supply.
    Bus       // RESET* -- the front-panel button. Warm.
};

virtual void reset(Reset) {}
virtual void power() {}
```

Conflating these is the classic source of "*works from power-on but not from the reset button.*"

Neither reset clears memory. Only removing power does. That is not a nuance, it is the rule, and the memory array is the proof: **a RAM chip has no POC* pin.** Its contents are indeterminate at power-up because *the chips just powered up*, not because a signal arrived. A `Reset::Bus` leaves memory intact and leaves media mounted and streams connected.

`power()` is the only thing that loses RAM and re-reads ROM images.

What each signal does is a fact about your board, and it comes from the manual. It is tempting to scrub the card clean on `RESET*` because it feels safe, and it is exactly backwards: **a card that resets more of itself than the reset line physically reaches is inventing a machine nobody built, and the invention is destructive.**

The 88-2SIO proves it. The **MC6850 has no RESET pin** — 24 pins, and RESET is not among them — so `RESET*` reaches that card's address decoding and *nothing else*. This tree used to reset both ACIAs anyway, throwing away the guest's word format and interrupt enables and eating a byte out of the receive register, on a card where a real bus reset would have preserved all of it.

The memory card is the model to copy: **it clears its bank latch on either reset and touches not one byte of RAM.**

Your board's `.md` must say concretely what each of the two does to it. That is a gate, not a nicety (DESIGN.md §14).

Lifecycle, time, and the host

`pump()` — the one door to the outside world

```
virtual void pump() {}
```

Give the host a turn: accept a socket connection, drain a keyboard. It is called **once per time slice by the run loop, and NEVER from inside a bus cycle.**

That seam is what keeps a board pure. `read()` and `write()` are pure computation over the card's state; anything that has to *talk to the outside world* happens here, at a known point in emulated time. That is what would let a recorded session replay identically instead of depending on when the host scheduler happened to deliver a packet.

Boards must **never** touch a socket, a file handle or `termios` directly. Everything they need from the host comes through the services in `src/host/` and `src/platform/`.

`Clock` — emulated time

Time is measured in **T-states**, never milliseconds, and it advances **only when the CPU retires an instruction, by exactly the count the CPU reported** (`StepResult::tStates`). `Machine::clock` is the one clock; a board is handed it by `attachClock()` when it goes into the backplane.

Nothing else in the simulator may call `std::chrono::now()`. That is what makes the guest's sense of time and a card's sense of time *the same sense of time* — a UART's idea of when a character has finished going out is derived from the very instruction stream the guest is timing it against, so the two cannot drift.

A card with nothing time-dependent on it never looks at the clock, and most don't. A UART absolutely does: **TDRE is a deadline, not a flag.** Two facilities, and they answer different questions:

- **`Clock::at()` / `after()`** — a *deadline*: something emulated time already knows is coming. A character finishing transmission. A sector arriving under the head.

- `pump()` — a *keystroke*, which is not in emulated time at all and which nothing could have scheduled.

The 6850 needs both, in the same function: `pump()` takes the byte off the line, and if the line has not yet had time to deliver it, sets a deadline for when it will.

The crystal is on the CPU card, so `clock_hz` is that board's property. There is no machine clock and there must never be one again — a machine-level copy would be a second place to say one thing, and the day the two disagreed the machine would run at whichever was written last.

`drainLog()` — what the card wants said out loud

```
virtual std::vector<std::string> drainLog() { return {}; }
```

A bank select it could not decode. A ROM that failed to load. A sector whose checksum did not match. Drained by the monitor after every command and after every run, and cleared by the draining; MCP returns the same strings as structured data.

It is on `Board`, and it is virtual, because it used to be a `dynamic_cast<MemoryBoard*>` in `Machine::drainBoardLog()` — which meant a disk controller with something to say about a bad sector **had no way to say it**. A general facility with one card's name compiled into it is a bug, every time.

A chip is not a card

`src/chips/` holds the parts that get soldered onto more than one card: `mc6850.h` (both halves of the 88-2SIO), `uart1602.h` (the COM2502 — the 88-SIO and the 88-ACR), `wd17xx.h` (the FD1771).

Each chip is modelled from its DATA SHEET. Each board is modelled from its MANUAL. A chip built instead from the one BIOS that happens to drive it implements exactly the subset that BIOS touches and quietly gets the rest wrong — and it looks finished while doing it.

A chip knows nothing about S-100. It has a clock, some pins, and (if it moves bytes) a `ByteStream`. What it talks to is an **interface, never a file**: `Wd1771` has a `FloppyDrive` — STEP, DIRC, TR00, WPRT, READY, IP, with a drive on the far end. It owns no image, no geometry and no host file, and it has **no drive-select and no side-select pin**, because the FD1771 hasn't got either. Which drive those pins reach is a **latch on the card**. That absence is the chip/board seam stated in silicon.

And the seam is not "the chip does the work and the board forwards to it." The 88-SIO's status word is **inverted** and the 88-2SIO's is not. A shared UART class with a `bool invert` on it

is *precisely* the bug `src/chips/` exists to prevent — so those two cards share **no code**, on purpose. What licenses sharing is being **the same part**, not filling the same role.

Where the seam falls is a fact about the chip, not a house style. The chip is what the data sheet describes, at its pins, in true sense. Everything between those pins and the S-100 bus — the inverting buffers, which bit each signal lands on, the interrupt-enable flip-flops that live in a *different IC*, the port decode — is the **card's**, and it stays on the card.

Where this leaves you

You now know what a board is: a thing that is asked four pure questions (`assertsPhantom`, `decodes`, `assertsInt`, `assertsVi`), that is *told* two things (`read`, `write`), that may watch the backplane (`snoop`), that describes itself (`properties`), and that gets a turn to talk to the host at one known point in emulated time (`pump`). Everything else on the vtable is a detail of one of those.

You also know the traps: a decode with a side effect in it, a decode that changed without saying so, a missing `intChanged()`, a card that resets more of itself than the real one does, and a board that manufactures its own `0xFF`.

That is enough to write a board. The next chapter writes one — a lamp latch on `OUT FF`, which is free precisely because the front panel decodes `IN FF` and nothing else.

Writing a board

We are going to build a card. A real one — it plugs into the backplane, it decodes a bus cycle, it holds state, it has a setting you can put in a machine file, and when we are done the monitor will know about it, `CONFIG SAVE` will write it out, and an AI assistant driving the machine over MCP will be able to configure it. None of that last part will cost us a line of code.

The card is **eight lamps and a latch**. `OUT 0FFH` lights them. That is the whole thing.

The finished source is in the tree at **`examples/boards/lamp/lamp.h`**, and `tests/test_lamp.cpp` drives it on a real bus — so it compiles and it is tested, which is not something you can say about most tutorial code. Read along, or write it yourself.

Why port FF, which looks taken

The front panel is already at port `0xFF` — that is where the SENSE switches live. So this looks like a collision, and picking `FF` looks like a mistake.

Ask the machine:

```
altairsim> WHO IO FF
port FF OUT: nobody (an OUT here goes nowhere)
```

```
altairsim> SHOW BUS IO
I/O
  FF        fp0      read  SENSE switches SA8..SA15 -> D0..D7
  10        sio0     read/write 6850 'a' -- status/control, data
  ...
```

The front panel answers `IN FF`. It does not answer `OUT FF`. That is not a simplification in the model — it is the real card. The SENSE switch buffer's enable is gated with `sINP`; there is no `sOUT` anywhere near it. On a real Altair an `OUT 0FFH` is not the panel's, and the byte goes nowhere at all.

And the bus knows the difference, because **the bus decodes each direction separately** — `ioRead_[256]` and `ioWrite_[256]` are two different tables (`src/core/bus.h`), for the same reason the real backplane has two different strobes.

> **The bus routes by cycle type, not just by address.**

So `OUT FF` is a genuinely empty slot in a stock machine, and our card can have it without disturbing anything. Keep that sentence in mind while writing `decodes()` — it is the single easiest thing to get wrong, and getting it wrong here would break the SENSE switches.

1. The card

`examples/boards/lamp/lamp.h`. A board is a class that inherits `Board` (`src/core/board.h`) and answers some questions.

Its name

```
std::string type() const override { return "lamp"; }
```

This is the word a machine file uses: `type = "lamp"`. It is the **chip or the common name**, never a catalog number — nobody ever asked for an 88-CPU, they asked for an 8080.

What it decodes

```
bool decodes(const BusCycle& c) const override {
    if (!enabled_) return false;                // a card switched off drives nothing
    if (c.type != Cycle::IoWrite) return false;  // OUT only. The panel owns IN.
    return c.port() == port_;
}
```

That middle line is the whole lesson of this chapter. Delete it and the card answers `IN FF` too — the SENSE switches stop working, and the bus starts reporting contention with `fp0`.

Two rules about `decodes()`, and they are not negotiable:

- **It must be pure and combinational.** It answers "*if this cycle happened, would I drive the bus?*" and it does nothing else. No side effects, no counters, no latching.
- **The bus caches the answer.** It keeps one slot per port and per page, so a decode is a table lookup rather than a walk over every card in the machine. Which is why a `decodes()` with a side effect in it is a bug that will not show up for a month: it does not get called when you think it does.

If a board's decode ever *changes*, it must say so — `decodeChanged()`. (You will see below that we get that for free.)

What it does with the cycle

```
void write(const BusCycle& c) override { latch_ = c.data; }
```

We claimed the cycle, so the byte is ours.

We never override `read()`. We never say yes to a read, so we are never asked one — and an `IN FF` from our port floats to `0xFF`, which is exactly what an output-only card does.

Power and reset

```
void reset(Reset) override { latch_ = 0; }
void power() override      { latch_ = 0; }
```

These are **two different events** and a card is entitled to treat them differently. `Reset::Bus` is the RESET* line — the button on the panel. `Reset::PowerOn` is the power coming up, and it is the only thing that loses RAM. For our lamps they mean the same thing: the lights go out. For a memory card they emphatically do not.

Its state — SNAPSHOT and RESTORE

A board writes its **runtime state** so `SNAPSHOT` can save it and `RESTORE` can put it back (`DESIGN.md` §13). Chain to the base — it handles `enabled_` — then read and write your own fields in the same order:

```
void serialize(StateWriter& w) const override {
    Board::serialize(w);
    w.u8(latch_);
}
void deserialize(StateReader& r) override {
    Board::deserialize(r);
    latch_ = r.u8();
}
```

`StateWriter` / `StateReader` (`core/statefile.h`) are explicit and little-endian — `u8`, `u16`, `u32`, `u64`, `boolean`, `str`, `blob`, and `raw` for a fixed array — so a snapshot reads back byte-for-byte on every target. Three rules decide what goes in:

- **Write runtime state, not configuration.** A snapshot is RESTOREd into a machine built from the same machine file, so your straps (the port, a baud rate, a jumper) are already correct — writing them would be a second copy that could disagree. Write the registers, latches, RAM and in-flight buffers that a running machine accumulates.

- **Do not write a host handle.** A `ByteStream`, a disk image, the `Display` — those are re-opened from the (unchanged) config. But bytes that are **not on the host yet** — an uncommitted write buffer, a tape's head position — *are* your state and *do* travel.
- **Never write a `Clock::Handle`.** The event queue does not survive a snapshot. If your card sets a deadline, **re-arm it in `deserialize()`** from the state you just read — call the same `refresh()` / `arm…()` you already call when a jumper moves. Store the deadline as an absolute T-state if you need the timing back; it stays valid because the clock's time travels with it.

Its settings — and this is the part that pays

```
std::vector<Property> properties() override {
    std::vector<Property> p;
    {
        Property x;
        x.name  = "port";
        x.help  = "the port this card latches. Write-only -- an IN here is not ours";
        x.kind  = Kind::Int;
        x.radix = 16;               // ON THE WIRE -> HEX. A port is a thing the 8080 sees.
        x.min   = 0;
        x.max   = 0xFF;
        x.get   = [this] { return Value::ofInt(port_); };
        x.set   = [this](const Value& v, std::string&) {
            port_ = (uint8_t)v.i();
            return true;
        };
        p.push_back(std::move(x));
    }
    {
        Property x;
        x.name  = "lamps";
        x.help  = "what the guest last wrote -- the eight LEDs";
        x.kind  = Kind::Int;
        x.radix = 16;
        x.get   = [this] { return Value::ofInt(latch_); };
        // NO SETTER.
        p.push_back(std::move(x));
    }
    return p;
}
```

This vector is the entire configuration layer. There is no schema file, no parser, no registration call, and nowhere else to declare anything. `SET`, `SHOW`, the TOML loader, `CONFIG SAVE`, the MCP tool schemas, tab completion, and the User Manual's generated board reference are all written **once**, against this, and know nothing about any particular card.

Two details worth stealing:

- **`radix = 16`**, because a port is a thing the processor can see. On the wire → hex; never on the wire → decimal. Get this wrong and `port = 10` in a machine file quietly means ten instead of sixteen. **It declares the class, not a display preference** — the one `radix` both reads bare digits and prints them, an operator can force any base of their own (`0x`, `0o`, `0b`, `#`) whatever you chose, and `SET CONSOLE base=octal` does not reach it: that switches how the *monitor* prints addresses and bytes, while `SHOW` prints every property in its own radix. So pick it by which side of the wire the number lives on, and never because one base reads better.
- **`lamps` has no setter**, and that absence *is* the signal. `SHOW` prints `(read-only)`, `CONFIG SAVE` leaves it out of the file, and the manual's reference marks it. A setter that merely *refused* would stop a `SET` and fool all three at once — which is a mistake that was in this codebase, on the memory card, until this manual went looking.
- **We never call `decodeChanged()`** in the `port` setter. The property layer calls it for us after any successful set, precisely so that a board author cannot forget.

What it tells the operator

```
std::vector<MapEntry> ioMap() const override {
    return {{port_, port_, "write", "lamp latch -- D0..D7"}};
}
```

This is what `BOARDS`, `SHOW BUS IO` and `WHO` print. **It is documentation, not decode** — the bus never consults it. A card whose `ioMap()` disagreed with its `decodes()` would work perfectly and lie to you, which is worse than a card that does not work.

2. Put it in the registry

Two lines and an include, in `src/boards/registry.cpp`:

```
#include "../../examples/boards/lamp/lamp.h"

// ...in boardTypes():
{"lamp", "A write-only latch: OUT FF lights eight LEDs. The Developer Guide builds this"},

// ...in makeBoard():
if (type == "lamp") return std::make_unique<LampBoard>();
```

That is all. `registry.h` promises "*adding a board type is one line here and nothing anywhere else*", and it means it.

(Our card is a header, so it needs no entry in `CMakeLists.txt`. A real one — with a `.cpp` — adds its source to the `altair_core` library alongside the others.)

3. Build it, and watch what you get for free

```
$ cmake --build build -j
```

The card is now in the catalogue. `SHOW BOARDS` lists every type with its description; name one to see its settings and their help text -- and nobody wrote a line of code to put it there:

```
altairsim> SHOW BOARDS
  TYPE        DESCRIPTION
  ----------  ------------------------------------------------------------
  ...
  lamp        A write-only latch: OUT FF lights eight LEDs. The Developer
              Guide builds this

  SHOW BOARD <type> for a board's properties

altairsim> SHOW BOARD lamp
  lamp  A write-only latch: OUT FF lights eight LEDs. The Developer Guide
        builds this

  PROPERTY  HELP
  --------  ----------------------------------------------------------------
  port      the port this card latches. Write-only -- an IN here is not ours
  lamps     what the guest last wrote -- the eight LEDs
```

Fit one, and ask the machine the same question we asked at the start:

```
altairsim> BOARDS ADD lamp lamp0
lamp0: lamp added

altairsim> WHO IO FF
port FF OUT: lamp0
```

It said `nobody` an hour ago. Now light the lamps:

```
altairsim> OUT FF 55
port FF <- 55

altairsim> SHOW lamp0
lamp0  (lamp)

  property         value            legal
  port             0xFF             0x0..0xFF
  lamps            0x55             (read-only)
```

`OUT FF 55` ran a **real output cycle on a real bus** — the same path the 8080's `OUT` instruction takes, because there is only one. The card decoded it, latched it, and `SHOW` read it back out of the reflection layer. `lamps` is marked read-only, and it worked that out from the missing setter.

And the front panel is still sitting on the same port, untroubled:

```
altairsim> IN FF
port FF -> 00      <- still the SENSE switches
```

One port. Two cards. No contention. **Because the bus routes by cycle type.**

4. Put it in a machine

```toml
[machine]
name = "lamps"
base = "default"

[[board]]
type = "lamp"
id   = "lamp0"
port = FF          # radix 16 -- no 0x needed
```

Nobody taught the TOML loader what a lamp is. It asked the card for its properties, found one called `port`, and set it. A board added next year is configurable, scriptable, drivable by an assistant, and documented **the day it lands**.

5. Test it

`tests/test_lamp.cpp` is the model. It does three things, and the middle one is the important one:

```
// A REAL OUT CYCLE on the bus -- not a method call on the board.
m.bus.ioWrite(0xFF, 0x55);
CHECK(lamp->properties()[1].get().i() == 0x55, "OUT FF latches the byte");

// The card is WRITE-ONLY, and the proof is that the read FLOATS.
CHECK(m.bus.ioRead(0xFF) == 0xFF, "an IN from FF is not the lamp's -- the bus floats");
```

…and then it pins down the claim this whole chapter rests on, with both cards in one machine:

```
BusCycle in {Cycle::IoRead,  0xFF};
BusCycle out{Cycle::IoWrite, 0xFF};

CHECK(m.bus.respondersTo(in).size()  == 1, "IN FF: the front panel, and ONLY the front panel");
CHECK(m.bus.respondersTo(out).size() == 1, "OUT FF: the lamp, and ONLY the lamp");
CHECK(m.bus.drain().empty(),               "...and the bus says NOTHING. It is not contention.");
```

Assert the thing your design depends on, not the thing that is easy to assert. If the bus ever stopped decoding by direction, this chapter would be teaching a falsehood — and that test is what would say so.

> **One trap, and it caught this test.** `m.bus.attach(board)` wires a card to the backplane; it does not hand it to the **machine**. Lifecycle events — `reset()`, `power()`, `pump()` — are dispatched by `Machine` over the boards it *owns*. A board that is only on the bus will answer cycles all day and never hear the RESET line. A card that comes from a machine file goes in through `Machine::add()` and gets both.

What to do next

Our card answers a cycle. A more interesting one **asks for something**:

- **Interrupts.** Add an `IrqJumper` and push `irqJumperProperty("interrupt", ..., irq_)` into `properties()` — that one call buys you the whole `none | int | vi0..vi7` vocabulary, tab completion, and the flag that lets `SHOW BUS IRQ` find your strap. Then override `assertsInt()` / `assertsVi()`, and **call `intChanged()` from every place your pending flag could move.** A spurious call costs a virtual call. A missing one hangs the guest forever.
- **Time.** A card with a deadline uses the `Clock` it was handed. A UART absolutely needs one: transmit-buffer-empty is a deadline, not a flag.
- **The outside world.** Anything that talks to a socket, a file or a keyboard does it in `pump()` — **never inside a bus cycle**. That seam is what keeps `read()` and `write()` pure computation over state, and it is what a deterministic replay would be built on. The **88-**

C700 printer (`src/boards/mits-88c700.{h,cpp}`, `docs/boards/mits-88c700.md`) is the smallest shipped card that does this for real: a bare latch that sinks its data port to a `ByteStream`, so `CONNECT lpt0:prn file:printout.txt` captures the printout and the card never learns what a file is. It is a good next read after this lamp.

- **Sub-units.** A card with a *list* of things — regions on a memory card, drives on a controller — declares `subUnitTables()` and gets `[[board.region]]` / `[[board.drive]]` in TOML for free.

And whatever you build: **it ships with its `.md`.** A board and its documentation are one deliverable, and the **Limitations** and **Quirks** sections are the load-bearing ones — they are what forces the honest question "*what did I not actually implement?*", which is exactly the question a simulator author most wants to avoid. `docs/boards/_TEMPLATE.md` is the form.